

HuGo: LLMs as Whole-Body Policy Code Designers for Humanoid Loco-Manipulation

Seoyeon Choi¹, Shizhao Ye^{1,2}, Nicholas Bui¹, Aayushi Shrivastava¹, Kanghyun Ryu¹, Dhruva Tirumala³, Markus Wulfmeier⁴, and Negar Mehr¹

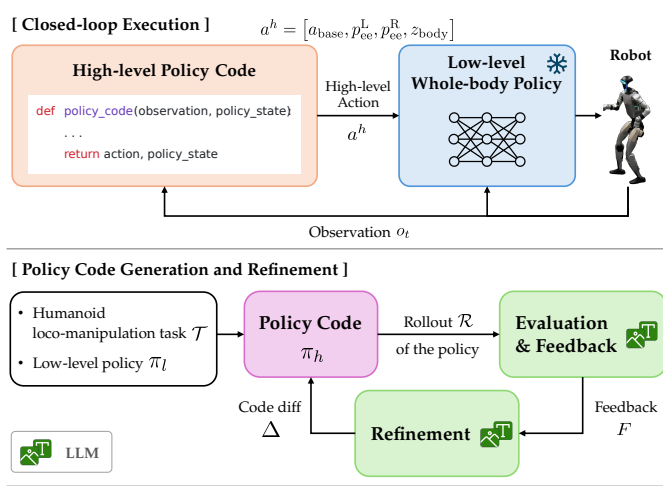


Fig. 1: We introduce **HuGo**, a framework for generating closed-loop policy codes for humanoid loco-manipulation from task descriptions, without expert demonstrations, reference motions, or reward design. (Top left) The generated code outputs a high-level command a^h at every timestep, consisting of planar locomotion, left and right end-effector positions, and body height, which a frozen low-level whole-body policy converts into joint actions. (Bottom left) The LLM generates the policy code, diagnoses the policy’s rollouts from numerical trajectories and selected video frames, and proposes a code diff, a localized update to the current policy rather than a full regeneration. (Right) Policies generated in simulation transferred zero-shot to hardware for Push Button (top) and Lift Box (bottom).

Abstract—For humanoids to be useful in everyday environments, they must perform a wide range of tasks that couple locomotion and manipulation. Existing approaches commonly acquire a loco-manipulation policy through reward engineering or demonstrations followed by task-specific training, making it costly to scale to new tasks. In this work, we propose a hierarchical approach to humanoid loco-manipulation that eliminates these per-task requirements. HuGo, Humanoid policy code Generation, uses a Large Language Model (LLM) to generate executable, closed-loop high-level policy code from a task description on top of a frozen low-level whole-body policy. Given the task, observation, and command specifications, the LLM constructs the task logic in code. HuGo then refines the policy from its rollouts using numerical trajectories and selected video frames to produce feedback and targeted code updates. Across five simulation tasks, using two different low-level policies, HuGo substantially outperforms a high-level reinforcement learning baseline and approaches the performance of a demonstration-based baseline. We achieve this

level of performance without task-specific reward design or demonstration collection. We further demonstrate zero-shot transfer of simulation-generated policies to hardware and show that applying the same refinement loop to real-world rollouts can further improve transfer performance without expert demonstrations or policy retraining. Project website is <https://iconlab.negarmehr.com/HuGo/>

I. INTRODUCTION

Recent advances in whole-body control have demonstrated what humanoid robots are capable of: they can now cartwheel, dance, and traverse rough terrain [1], [2]. Yet some of the most useful things we would ask them to do are less spectacular: open a door, press an elevator button, or lift a box onto a shelf. These everyday tasks require a policy that coordinates locomotion and object interaction, deciding where to stand, how to reach, and when to move.

Obtaining such a policy remains a key challenge in humanoid loco-manipulation. Reinforcement learning (RL) optimizes policies against a reward function [3]–[6], but can require substantial reward design [7] and repeated training to obtain the desired behavior. Demonstration-based methods guide learning with examples collected through teleopera-

*This work was not supported by any organization

¹University of California, Berkeley. {seoyeon99, nicholasqbui, aayushis, kanghyun.ryu, negar}@berkeley.edu

²ShanghaiTech University. yeshzh2023@shanghaitech.edu.cn

³Google DeepMind. dhruva92@gmail.com

⁴Nomagic. mwulfmeier@nomagic.ai. This work was conducted in part while the author was affiliated with Google DeepMind.

tion [8] or extracted from human videos [9], [10], introducing the effort of collecting suitable data and, for human motions, retargeting them to the robot. Either way, the cost recurs for every task we want the humanoid to perform, as another reward to shape or another set of demonstrations to collect.

Recently, Code as Policies and subsequent work [11]–[13] have shown that Large Language Models (LLMs) can generate robot policy codes from natural-language instructions and produce manipulation policies without any training. We ask whether humanoid loco-manipulation can benefit in the same way. Directly controlling the joints of a high-dimensional robot such as a humanoid with an LLM is impractical [14], [15]. However, existing whole-body policies can already track locomotion, end-effector, and body height commands [16], [17]. Our key insight is to have the LLM generate high-level policy code that outputs these commands, while reusing the frozen whole-body policy, avoiding motion data, reward engineering, and retraining for each new task.

To this end, we introduce **HuGo**, Humanoid policy code Generation, which turns a task description into a closed-loop high-level policy *without any task-specific training*. We prompt an LLM with the task description, the observations available to the robot, and the commands its low-level policy accepts, and have it generate policy code that reads the observation at every timestep and outputs the next command. *Note that we provide the LLM with no example policies, no reference motions, and no library of task skills*, leaving the task logic entirely to the LLM. A policy generated from a task description is unlikely to be correct on the first attempt, so the LLM evaluates the execution of the policy from its numerical trajectory and selected video frames, and refines the code from that diagnosis. This refinement loop takes as input nothing but executed trajectories, and can therefore work in both simulation and on real hardware.

We validate HuGo across five tasks and two low-level policies, in both simulation and on hardware. The generated policies substantially outperform a reinforcement learning baseline trained with hand-designed rewards, and approach the performance of a demonstration-based baseline on its own tasks. Policies generated in simulation can transfer zero-shot to hardware, and for a policy whose success drops after transfer, three refinement depths carried out directly on the physical robot raise it from 20% to 90%.

In summary, our contributions are as follows:

- 1) We propose HuGo, a framework in which an LLM generates closed-loop high-level policy code for humanoid loco-manipulation from a task description, on top of a frozen whole-body policy. This requires no expert demonstrations, reference motions, or task-specific reward design.
- 2) We introduce a refinement loop that analyzes policy rollouts from numerical trajectories and selected video frames, and turns the resulting feedback into targeted code diffs.
- 3) We validate HuGo across five loco-manipulation tasks and two low-level policies in simulation and on hardware, including zero-shot sim-to-real transfer and policy refinement performed directly from hardware rollouts.

II. RELATED WORKS

A. LLM-Guided Optimization

As the reasoning ability of LLMs grows, recent work treats them as program optimizers guided by an evaluator [18]–[21]. In robotics, LLMs have been used to generate rewards [7], curricula [22], or multi-agent coordination feedback [23], but still rely on costly task-specific RL training. VLMs have also been used as the evaluator itself, judging whether a rollout succeeded from its video frames alone [24]. Following this line of work, we also use an LLM as an optimizer, but one that updates the policy code directly, guided by evaluations of the policy’s rollouts.

B. LLM-Generated Programmatic Robot Policies

A more recent line of work uses the LLM to write the policy itself. Code as Policies [11] first showed that an LLM can produce executable robot policies composing perception and control primitives. CaP-X [12] benchmarks such LLMs on manipulation, where the model acts as a connection between specialized modules such as grasping and motion planning, and later work organizes these calls as a graph of skill nodes [13] or improves the program across attempts by rollout fitness [25]. On humanoids, LLMs have produced behavior trees [26] and sequences of locomotion and manipulation primitives [27]. In all of these, the generated program is assembled from a library of skills and modules supplied to the model. We instead provide no primitives or modules to call. Given only descriptions of the policy observations and low-level policy’s command interface, the LLM must decide where to stand, where to reach, and when to transition between stages. The resulting code runs as a closed-loop policy evaluated at every timestep.

C. Humanoid Loco-Manipulation

Humanoid loco-manipulation is commonly split into a low-level policy that tracks whole-body commands [16], [17], and a high-level policy that decides what to command. The high-level policy is often obtained with reinforcement learning, through residual learning over a pretrained motion-tracking policy [3], blended pretrained skills [4], or MPC-guided training [5]. Others learn from human video priors [9], [10], collect demonstrations via MPC [28], or plan contact sequences for trajectory optimization [29]. Each new task, however, carries its own cost: a reward function, a demonstration and its retargeting, or a task-specific optimization problem. We instead obtain the high-level policy from a task description, on top of an unmodified low-level policy.

III. PROBLEM FORMULATION

We consider a humanoid performing a loco-manipulation task $\mathcal{T}$ with a hierarchical structure: a high-level policy π_h and a frozen low-level policy π_l . Our goal is to construct a task-specific π_h that designs a closed-loop strategy for the task, and outputs commands that π_l converts into joint-position targets.

We define a task as $\mathcal{T} = \langle \ell_{\text{task}}, \mathcal{X}, \rho, S \rangle$, where ℓ_{task} describes the task in natural language, $\mathcal{X}$ is the state space of

the humanoid, task objects, and the goal, ρ is the distribution over initial states $x_0 \in \mathcal{X}$, which randomizes the initial robot, object, and goal poses, and S is a binary success function indicating whether a rollout completes the task.

We assume that the high- and low-level policies are connected through a high-level command a^h passed to π_l at each timestep. We consider low-level policies that accept commands of the form:

$$a^h = [a_{\text{base}}, p_{\text{ee}}^L, p_{\text{ee}}^R, z_{\text{body}}] \in \mathbb{R}^{10}, \quad (1)$$

where $a_{\text{base}} \in \mathbb{R}^3$ specifies planar locomotion and heading, $p_{\text{ee}}^L, p_{\text{ee}}^R \in \mathbb{R}^3$ indicate the desired positions of the left and right end-effectors, and $z_{\text{body}} \in \mathbb{R}$ specifies the desired robot height. The command limits and coordinate frames are determined by π_l , and we provide them to π_h , which must produce commands consistent with them.

At each timestep t , the closed-loop high-level policy π_h receives an observation o_t and produces the high-level command a_t^h . The observation is $o_t = [o_t^{\text{robot}}, o_t^{\text{task}}, a_{t-1}^h]$. Here, o_t^{robot} contains the base position and heading angle in the world frame, together with the measured base velocity, end-effector positions, and body height, while o_t^{task} contains task-relevant object and goal information, such as their poses, and a_{t-1}^h is the previous high-level command. The observation o_t consists of numerical quantities rather than raw sensor data such as images. We assume it is available at each timestep, whether from simulation, external tracking, or vision.

We evaluate a high-level policy π_h by executing it with the frozen low-level policy π_l from an initial state $x_0 \sim \rho$. Let $\mathcal{R}(\pi_h, \pi_l; x_0)$, written $\mathcal{R}$ for brevity, denote the resulting rollout. Our objective is to find the high-level policy π_h that maximizes the expected task success: $\pi_h^* = \arg \max_{\pi_h} \mathbb{E}_{x_0 \sim \rho} [S(\mathcal{R}(\pi_h, \pi_l; x_0))]$.

IV. METHOD

To obtain a high-level policy π_h , we leverage the reasoning and task understanding of an LLM. However, an LLM cannot be queried at the rate a humanoid must be controlled, whereas the code it writes can run at that rate. Therefore, we represent π_h as an *executable policy code* that may maintain an internal state m_t across timesteps:

$$(a_t^h, m_t) = \pi_h(o_t, m_{t-1}). \quad (2)$$

The internal state m_t allows π_h to retain task-relevant information across timesteps. Its structure is defined by the generated code and initialized at the beginning of each episode. In a Push Button task where the humanoid walks toward a button and presses it, for instance, m_t may be a discrete phase that moves from approaching, to aligning, to pressing, or the commands issued at the previous timestep so that they can be smoothed. In the following subsections, we describe in detail how our method generates π_h .

A. Zero-shot Policy Generation

For a given task $\mathcal{T}$, the generated policy code must specify a closed-loop strategy for the entire execution. In a Push Button task, the button appears at a different position,

height, and facing direction in each episode, so a fixed sequence of commands may not be able to press it. The code must instead determine each command from the current observation, inferring an approach position from the button pose, deciding when to stop walking, and commanding the end-effector position and robot height needed to press it.

We provide the LLM with a natural-language prompt as $p = [\ell_{\text{task}}, \ell_{\text{robot}}, \ell_{\text{obs}}, \ell_{\text{act}}]$, where ℓ_{task} describes the task, ℓ_{robot} provides relevant robot information, such as its pelvis-to-head offset and arm length, and ℓ_{obs} and ℓ_{act} describe the observations available to π_h and the high-level commands accepted by π_l , including their limits and coordinate frames. This prompt remains fixed throughout the pipeline. Importantly, *it contains no demonstrations, example policy codes, action sequences, or reference implementations*. Given the prompt p , the LLM generates the zero-shot policy code π_h^0 . The prompt alone, however, does not convey how these commands interact with π_l , the robot, and the environment, so policy logic that appears reasonable may still fail in execution due to tracking errors, contact dynamics, or poorly chosen transition conditions. We therefore examine how π_h^0 behaves through its rollouts, and diagnose the behavior.

B. Rollout Evaluation and Feedback

We evaluate the current policy across M initial states and measure its success rate using S in simulation. This success rate shows how well the policy performs, but not why it succeeds or fails. Therefore, we record the rollouts and select the first unsuccessful rollout when it is available, since failures provide direct evidence about flaws in the policy logic. The selected rollout consists of $\mathcal{R} = (o_t, m_t, I_t)_{t=0}^T$, where I_t is the video frame at timestep t and T is the termination timestep.

Selecting what to show. The observation covers the robot’s base and end-effector position, and the object poses, but not the full body configuration. Whether the humanoid leans off balance as it reaches for the button, for example, is invisible in the numbers but immediate in an image, so we provide the images to the LLM. A rollout spans thousands of timesteps, far more frames than can fit in context, and the relevant moments vary across tasks and policies. We therefore use the LLM to select task-relevant keyframes.

Analyzing the rollout. Given the prompt p , the current policy code, and a sparse sample of o_t and m_t from $\mathcal{R}$, the LLM generates an executable analysis function f_{analyze} . This function is generated specifically for the current rollout and policy, rather than fixed in advance. Applied to the complete trajectory, it returns a numerical summary D and task-relevant keyframe indices k_{key} . The summary is a string of at most 200 words containing quantities and events that the LLM determines may be useful for evaluating the rollout, such as distance extrema, threshold crossings, tracking errors, and internal-state transitions. Keyframes alone [30] may omit the broader progression of the rollout, while evenly sampled frames [23] may miss brief events such as a contact. We therefore combine both under a fixed frame budget B : when

f_{analyze} returns fewer than B keyframes, the remaining slots are filled with evenly spaced frames, giving the indices k .

Generating feedback. The LLM receives the prompt p , the current policy code, the summary D , and the trajectory samples and video frames at k . From this, the LLM returns natural-language feedback F describing what occurred during execution and the likely cause of success or failure. Note that this feedback rests on a single rollout and has no access to the scalar success rate of the current policy.

C. Iterative Policy Code Refinement

Given the rollout feedback F , rather than regenerating the entire policy, we have the LLM propose a localized code diff Δ conditioned on the current policy code. This preserves policy logic that works while modifying only the parts that F identifies as a fault, and keeps the generated output concise.

Such a modification is not guaranteed to improve the policy, so refinement proceeds over multiple attempts. Note that failed attempts are still informative. Providing the LLM with previously attempted modifications and their evaluations helps it avoid repeating them. However, accumulating every attempt across the entire process would cause the context to grow continually. We therefore keep this history of failed attempts only while it remains relevant, that is, while the LLM is still trying to improve the same policy.

We organize refinement into *depths* and *rounds*. A depth is one improvement over the policy it starts from, and it is given up to three rounds to find one. In each round, the LLM proposes a diff, and the resulting candidate is evaluated as in Sec. IV-B. If the candidate reaches a higher success rate, it is accepted: it becomes the current policy, the history is cleared, and the next depth begins from it. If not, its diff, success rate, and feedback are added to the history, and the next round proposes a different modification to the same policy.

Refinement terminates when either the policy reaches 100% success, or when the maximum refinement depth D_{max} is reached, or when all three rounds in one depth fail to improve the current policy. We stop after three failed rounds because the task logic a policy code encodes cannot always be repaired by incremental changes. In all cases, we return the current best-performing policy as the final refined high-level policy. Algorithm 1 summarizes the complete procedure, from zero-shot generation through refinement, that yields a single high-level policy.

D. Best-of- N Selection

LLM generation is stochastic, so independent runs produce policies with qualitatively different task logic. Refinement edits a policy locally and rarely replaces the logic it starts from, so the final performance depends on that initial logic. Therefore, we run the full pipeline N times, each from a new zero-shot policy, and keep the one with the highest success rate. Even repeated N times, the procedure requires no human intervention and costs on the order of ten dollars and a few hours, as we show in Sec. V-D.

Algorithm 1 Generation of a Single High-Level Policy

Require: Prompt p , frozen low-level policy π_l , M initial states, maximum refinement depth D_{max}
Ensure: Refined high-level policy π_h

- 1: Generate zero-shot policy π_h from p
- 2: Evaluate π_h over M initializations to obtain success rate s
- 3: **for** $d = 1, \dots, D_{\text{max}}$ **do**
- 4: **if** $s = 100\%$ **then return** π_h
- 5: Diagnose a rollout of π_h as in Sec. IV-B to obtain feedback F
- 6: Initialize the refinement context with p , π_h , and F
- 7: **for** $r = 1, 2, 3$ **do**
- 8: Generate code diff Δ from the refinement context
- 9: Apply Δ to π_h to obtain candidate $\tilde{\pi}_h$
- 10: Evaluate $\tilde{\pi}_h$ over M initializations to obtain success rate $\tilde{s}$
- 11: **if** $\tilde{s} > s$ **then**
- 12: $\pi_h \leftarrow \tilde{\pi}_h$, $s \leftarrow \tilde{s}$
- 13: **break**
- 14: **else if** $r = 3$ **then**
- 15: **return** π_h
- 16: **else**
- 17: Diagnose a rollout of $\tilde{\pi}_h$ to obtain feedback $\tilde{F}$
- 18: Add Δ , $\tilde{s}$, and $\tilde{F}$ to the refinement context
- 19: **end if**
- 20: **end for**
- 21: **end for**
- 22: **return** π_h

V. SIMULATION EXPERIMENTS

A. Experiment Setup

We evaluate our method on five humanoid loco-manipulation tasks shown in Fig. 2. These tasks are situated in an indoor environment where the surrounding walls constrain the available paths and require the policy to account for obstacles during navigation:

- Low Gate Passing (LGP): The humanoid must pass through two low gates by lowering its body height.
- Narrow Gate Passing (NGP): The humanoid must pass through two narrow gates by turning its body sideways.
- Push Button (PBT): The humanoid must approach a button and press it using one of its end-effectors.
- Push Box (PBX): The humanoid must approach a box and push it toward a target within 10 cm and hold it there for one second.
- Lift Box (LBX): The humanoid must approach a box resting on a podium, grasp it, and lift it above 1.0 m.

In every task we randomize the initial poses of the humanoid, objects, and goals. This changes the navigation path, approach pose, and timing of the required commands, requiring the policy to adapt accordingly. These tasks also pose distinct challenges: the gate-passing tasks emphasize navigation and whole-body configuration through constrained spaces, whereas the remaining tasks require the policy to coordinate navigation with precise object interaction.

We evaluate our method using two frozen low-level policies with different command specifications, demonstrating our method’s compatibility with different low-level policies:

- **RL-based policy** (r1isaac): We train a low-level policy using dense rewards for tracking the high-level commands. The base command specifies planar linear and yaw velocities, the end-effector targets are expressed in the torso frame, and the height specifies the desired torso height. The reward design is adapted from low-level policy in [31].

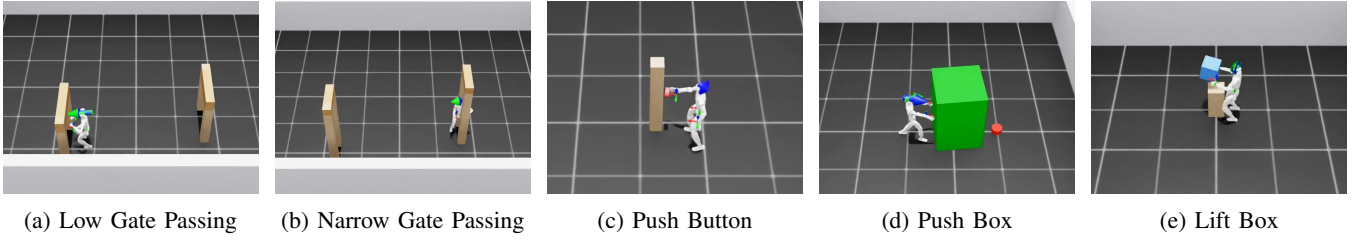


Fig. 2: The five simulated loco-manipulation tasks. All are situated in an indoor environment and randomize their initial conditions, so a policy must adapt its navigation path, approach pose, and command timing to each episode.

Low-level	Method	Low Gate Passing	Narrow Gate Passing	Push Button	Push Box	Lift Box
rl.isaac	High-level RL	11.33 ± 1.70	0.0 ± 0.0	7.33 ± 1.70	1.67 ± 1.25	0.0 ± 0.0
	HuGo (no refinement)	100.00 ± 0.00	67.15 ± 14.13	73.92 ± 7.56	49.31 ± 17.49	44.62 ± 25.22
	HuGo	100.00 ± 0.00	96.20 ± 1.31	79.97 ± 1.68	89.33 ± 11.37	75.92 ± 16.44
sonic	HuGo (no refinement)	-	65.93 ± 21.98	48.70 ± 16.43	46.30 ± 25.86	11.00 ± 3.79
	HuGo	-	97.54 ± 1.97	85.78 ± 7.89	61.79 ± 12.18	24.91 ± 6.12

TABLE I: Success rates (%) on the five-task suite with best-of-10 selection. Reported success rates are evaluated on 100 held-out initial states not used for policy refinement or best-of-N selection. The same held-out initial states are used for all methods. HuGo (no refinement) selects among the same policies before refinement. sonic is not evaluated on LGP as it cannot change its height while walking.

- **SONIC 3-Point VR (sonic):** We use the 3-point VR controller from SONIC [17], a state-of-the-art motion-tracking policy trained on a large-scale human-motion dataset. The base command specifies planar linear velocity and a yaw heading, the end-effector targets are expressed in the pelvis frame, and the height specifies the desired pelvis height. Since SONIC cannot change its height while walking, we do not evaluate it on LGP.

Baselines. We compare against two alternative ways of obtaining a high-level policy. **High-level RL:** a high-level policy trained with PPO [32] for 384M environment steps, using the same observations, command space, and low-level policy as ours, with a hand-designed dense reward for each task. **HDMI** [9]: a demonstration-based method that learns interactive humanoid whole-body control by imitating reference motions extracted from human videos. We evaluate against HDMI on two of its tasks that differ in where the difficulty lies, one in navigation and other in manipulation:

- Push Door with Hand (PDH): The humanoid must approach a door, push it open, and walk through.
- Carry and Place Bread Box (CPB): The humanoid must grasp a bread box, carry it, and place it at a target.

These tasks fix the initial pose of the humanoid relative to the object. For CPB, we reduce the bread box mass to a payload value supported by our low-level policies.

Implementation details. All experiments use a Unitree G1 in IsaacSim on a single NVIDIA L40S GPU. The generated policy code runs at 50 Hz with rl.isaac and at 10 Hz with sonic, matching each deployment stack. We use gpt-5.4 for the LLM, and each policy is evaluated on a fixed set of $M = 100$ initial states. The reported final results use a separate held-out set. The evaluator receives $B = 10$ frames per rollout. Refinement runs for at most $D_{\max} = 7$ depths with three rounds each. For each task, we independently generate and refine 30 policies, and report best-of-10 results by averaging over 2000 random subsets of

	PDH	CPB
HDMI [9]	99.95 ± 2.21	98.4 ± 12.6
HuGo	100.0 ± 0.00	81.25 ± 26.23

TABLE II: Success rates (%) on two HDMI tasks. HDMI trains a policy per task from a human demonstration, while our policies are generated from a task description without demonstrations. HuGo is reported with best-of-10 selection.

10 of these policies.

B. Main Results

We report success rates on our five-task suite in Table I. With the rl.isaac low-level policy, our method achieves near or above 80% success on four of the five tasks, including near-perfect success on both gate-passing tasks. Our method also works with sonic without any change to the pipeline since switching low-level policies only requires describing the new command interface in the prompt. LBX remains the most challenging task, and the hardest with sonic, for reasons we examine in Remark. *Note that all of these policies are obtained from a task description, without a single expert demonstration, motion reference, or reward function.*

Refinement is essential to these results. Compared with selecting among policies before refinement, refinement improves the success rate by up to 40 percentage points. Refinement also substantially reduces the standard deviation in Table I for most of the tasks as it raises the success rate of policies across runs rather than of a few outliers. Fig. 3 shows how this improvement accumulates over refinement depths: most of the gain occurs within the first two depths, while harder tasks such as LBX continue to improve until the final depth.

Remark. LBX exposes a limitation in the LLM’s diagnosis of contact failures. The generated code often moves the end-effectors directly to the grasp width without first establishing clearance around the box, causing them to sweep against its sides. rl.isaac tracks accurately enough that

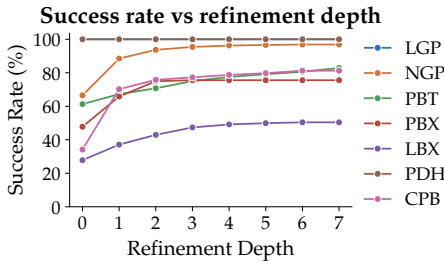


Fig. 3: Best-of-10 success rate versus refinement depth, averaged over low-level policies. Depth 0 corresponds to the policies before refinement.

	NGP	PBT	PBX	LBX	Avg.
Zero-shot	0.0	1.0	0.0	0.0	0.25
Text Only	47.0 $\pm$ 38.8	53.2 $\pm$ 23.9	19.6 $\pm$ 36.7	1.4 $\pm$ 2.3	30.3
Even Only	52.4 $\pm$ 42.9	55.4 $\pm$ 29.3	1.8 $\pm$ 1.3	13.0 $\pm$ 18.2	30.6
Keyframe Only	56.6 $\pm$ 46.3	45.6 $\pm$ 28.3	5.4 $\pm$ 6.2	20.2 $\pm$ 16.0	32.0
Ours	56.8 $\pm$ 46.4	75.8 $\pm$ 4.6	42.4 $\pm$ 34.1	35.6 $\pm$ 31.9	52.6

TABLE III: Success rates (%) after refinement when the rollout evaluation is given different information, mean $\pm$ standard deviation over five runs per task using `rl_isaac`. All variants start from the same zero-shot policies, shown in the first row.

they only brush the box, while `sonic`, with 4cm tracking error, pushes it off the podium. Although the LLM identifies the box being pushed off in evaluation, its feedback does not attribute this to the arm approach, so refinement never targets it. Adding a short geometry hint, without changing the interface or the low-level policy, raises the best-of-10 success rate with `sonic` from 24.9% to 72.3%, which is comparable to the `rl_isaac` results. The existing policy interface can therefore express an effective correction, but the LLM does not reliably infer it from rollout feedback without task-specific guidance. Another limitation comes from both low-level policies, which struggle to maintain the contact forces needed to hold the box, so it slips out during lifting.

Comparison with High-Level RL. Table I reports the high-level RL baseline over three training seeds. The trained policies remain at low success rates, far below the policies our method generates. Their failures are informative. `PBX`, for instance, requires the box to stay on the target for a full second, and the trained policy does bring the box to the target but commonly overshoots and pushes it straight off again. Across tasks, reinforcement learning discovers how to act on the object but not the staged structure the task requires, approach, align, act, and hold, which is exactly what our generated policy code specifies explicitly.

Comparison with Demonstration-Based Methods. Demonstration-based methods are a popular way to obtain humanoid loco-manipulation skills. We ask whether `HuGo`, which receives a task description, can approach the performance of methods that are directly trained to mimic a given demonstration. Table II reports the results on `PDH` and `CPB`. On `PDH`, our method matches `HDMI`, with both succeeding in essentially all episodes. On `CPB`, our method reaches 81.25% from a task description without demonstrations,

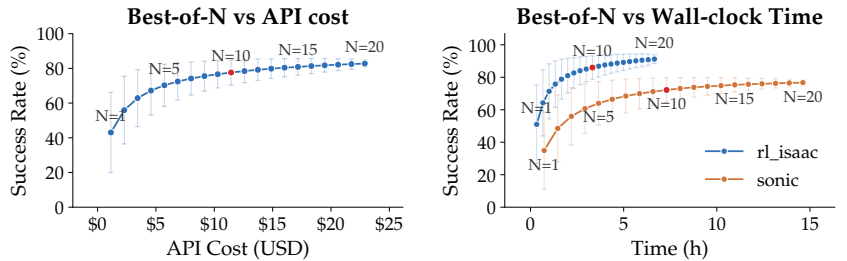


Fig. 4: Best-of- N success rate against the cost of obtaining it. Left: API cost, averaged over the five tasks and both low-level policies, where error bars denote the standard deviation across runs and the red marker indicates the operating point $N = 10$. Right: wall-clock time, shown separately for each low-level policy.



Fig. 5: PBT on hardware, all from the same policy code. The button position and facing direction determine which side the policy approaches from and which end-effector it presses with.

against 98.4% for `HDMI`, which imitates a single retargeted human demonstration of the task. The failures on `CPB` come from the contact-force limitation described above. `rl_isaac` tracks the end-effectors accurately enough for the approach to succeed, but neither low-level policy maintains force on a grasped object, so the box is frequently lost while being carried.

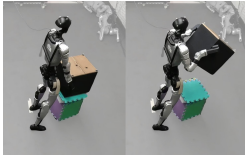
C. Variation of Performance

Policies generated independently for the same task differ widely in quality. Averaged over the five tasks and both low-level policies, a single generated policy (Best-of-1) reaches 43.9% success with a standard deviation of 23.4. This variation arises mainly because a task usually admits several sensible task logics that do not perform equally. On `PBX`, for instance, one recurring logic drives the box along the line joining the humanoid, the box, and the target, while another aligns with one face of the box and switches faces whenever the box drifts, which avoids rotating the box but rarely completes the task within the episode limit.

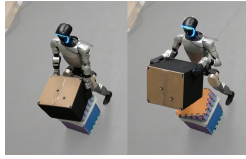
This diversity is a direct consequence of providing no examples and no hints about how a task should be solved. While it costs some consistency across runs, it is also what lets the method discover solutions instead of reproducing prescribed ones, and we are free to keep whichever one performs best. Such diversity is valuable in robotics, where the strategy that works best is rarely known in advance and often depends on the environment setup.

D. Cost of Reliability

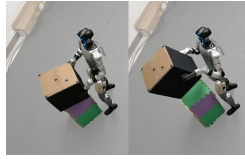
The variation described above means that a single generated policy is unreliable, which is why we perform best-of- N selection. Fig. 4 shows what this selection achieves against its API cost and its wall-clock time. Increasing N both raises



(a) Height 0.4m



(b) Height 0.5m



(c) Height 0.62m

Fig. 6: LBX on hardware, all from the same policy code. The policy sets its body height and end-effector targets from the height of the box rather than replaying a fixed motion.

the expected success rate and reduces its variation. Averaged over the five tasks and both low-level policies, the mean success rate rises from 43.9% to 79.1% from $N = 1$ to $N = 10$, while its standard deviation falls from 23.4 to 6.6. Best-of- N selection substantially improves reliability.

API cost. Generating and refining a single policy costs approximately \$1.1, so obtaining a best-of-10 policy costs approximately \$11. The curve increases monotonically but flattens beyond $N \approx 8$, so we select $N = 10$ as our operating point. Therefore, obtaining a policy for a new task costs on the order of ten dollars, which is negligible compared to the cost of designing rewards or collecting expert demonstrations.

Wall-clock time. We report wall-clock time separately for each low-level policy. Most of it is spent simulating rollouts rather than querying the LLM, and the two policies differ only because sonic plans one environment at a time while `rlisaac` can be evaluated as a batch. Obtaining a policy still takes a few hours, whereas the high-level RL baseline takes more than two days to train and needs a hand-designed reward for every task. Note that two days covers a single training run. Any change to the reward starts it over.

E. Ablation Studies

The quality of our refinement depends on what the LLM sees when it evaluates a rollout, namely the video frames and the numerical trajectory. To measure how each of these contributes, we select zero-shot policies with near-zero success rate for each task other than LGP, whose zero-shot policies already succeed, and repeat the refinement while varying only what the LLM receives. Table III reports the results.

Is visual information necessary? Since the numerical trajectory already contains the observations and internal states of the policy code, one may ask whether the video frames are needed at all. **Text Only** provides the numerical trajectory alone. Removing the frames cuts the success rate averaged over tasks by more than 40%, showing that visual information is necessary. Numbers alone do not reveal how the humanoid is positioned at the moment of contact with the object, which is precisely what distinguishes a policy that approaches the object correctly from one that does not.

Does it matter which frames we show? Having established that the visual frames are needed, we next ask how they should be chosen. **Even Only** uses evenly spaced frames throughout the episode, and **Keyframe Only** uses only the keyframes selected by the LLM without supplementing them with evenly spaced frames. Both perform worse than combining the two, confirming that the evaluation requires

Policy	PBT		LBX	
	Sim	Real	Sim	Real
1	90	93.3	78	66.7
2	90	20.0	70	20.0
3	79	73.3	64	6.7

TABLE IV: Success rates (%) of the three policy codes per task with the highest simulation success rate, evaluated in simulation (100 episodes) and on hardware (15 trials).

broad temporal coverage as well as the specific moments at which the task succeeds or fails.

VI. HARDWARE EXPERIMENTS

A. Hardware Setup

We deploy our high-level policies on a Unitree G1 humanoid using the same sonic low-level policy as in simulation. The observation o_t is constructed from a motion capture system. The observation and command specifications match those used in simulation, thereby allowing the generated policy code to run on hardware without modification.

B. Zero-Shot Transfer of Simulation-Generated Policies

A policy that succeeds in simulation is meaningful if it also succeeds on the physical robot, where contact, friction, and command tracking all differ from the simulation. We therefore evaluate zero-shot transfer on LBX and PBT, the two tasks that fit within our hardware workspace. For LBX, we use policies generated with the hint in Sec. V-B. For each task, we deploy three policy codes with the highest simulation success rate and run 15 trials per policy, using the same initial conditions for every policy. Table IV reports the results.

The best policy for each task transfers well zero-shot, succeeding in 93.3% of trials on PBT and 66.7% on LBX, close to its simulation success rate. Fig. 5 and Fig. 6 show the best-performing PBT and LBX policies running on the robot across different initial conditions, which each policy handles without any change between trials. However, zero-shot transfer is not equally successful for every policy. Some of the policies lose part of their success rate on hardware.

These losses are primarily associated with mismatches in low-level tracking and contact dynamics between simulation and hardware. The low-level policy tracks small planar velocity commands even less reliably on the hardware than in simulation. On LBX, the contact-force limitation we observe in simulation becomes more pronounced, and the box slips out of the end-effectors more often during the lift. A low-level policy better suited for contact-rich interaction could address this limitation without changing our high-level policy generation framework.

C. Refinement on Hardware

The zero-shot transfer results show that simulation-generated policies can transfer successfully to hardware, but may still suffer from sim-to-real discrepancies. Our refinement loop requires only rollouts of the current policy and can be applied directly on hardware without changing the prompts, evaluation, or refinement logic.

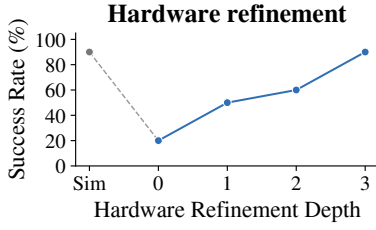


Fig. 7: Success rate at each refinement depth on hardware, starting from the policy that transfers worst. The simulation success rate of the same policy is shown for reference.

We demonstrate this on the PBT policy that transfers worst, which drops from 90% in simulation to 20% on hardware. Starting from this policy, we run the refinement loop on hardware, where the LLM analyzes the real-world rollouts to diagnose failure modes that emerge after transfer. Each candidate is evaluated over 10 trials. As shown in Fig. 7, the success rate increases at every refinement depth, from 20% to 50%, 60%, and finally 90%, matching the success rate the policy achieved in simulation. For this policy, three refinement depths, which required four rounds of refinement in total, recover its simulation performance.

The refinement identifies a hardware-specific failure mode: navigation succeeds, but the end-effector under-reaches the button. Fig. 8 shows the refinement from depth 1 to depth 2, where the feedback identifies a few-centimeter undershoot and adds an inward margin to the press target. At depth 3, the evaluation reports that the end-effector still falls short in some trials, and the refinement enlarges the same margin and holds the press longer, reaching 90%. The simulation does not exhibit this undershoot, so refinement in simulation would not have produced this correction.

This recovery is achieved entirely from hardware rollouts, using the same refinement pipeline as in simulation and without real-world expert demonstrations or policy retraining. This demonstrates that HuGo can correct sim-to-real discrepancies through refinement from hardware rollouts.

VII. CONCLUSION

We introduce HuGo, which generates closed-loop high-level policy code for humanoid loco-manipulation on top of a frozen whole-body policy, without expert demonstrations, reference motions, or reward design. The LLM generates the policy code and refines it using feedback from the policy’s rollouts. Across five tasks and two low-level policies, the generated policies adapt to randomized initial conditions, and support zero-shot transfer to hardware. We further demonstrate that the same refinement loop can correct sim-to-real discrepancies from hardware rollouts, raising a Push Button policy from 20% to 90% success.

Our method inherits the limits of the low-level policy and the LLM. A low-level policy that maintains contact force, or one with a dexterous hand, would raise success on tasks that require holding an object. The LLM sets the second ceiling, since the policy code can express a collision-free approach that the LLM does not infer on its own. For future work, a cross-task memory would let lessons from earlier

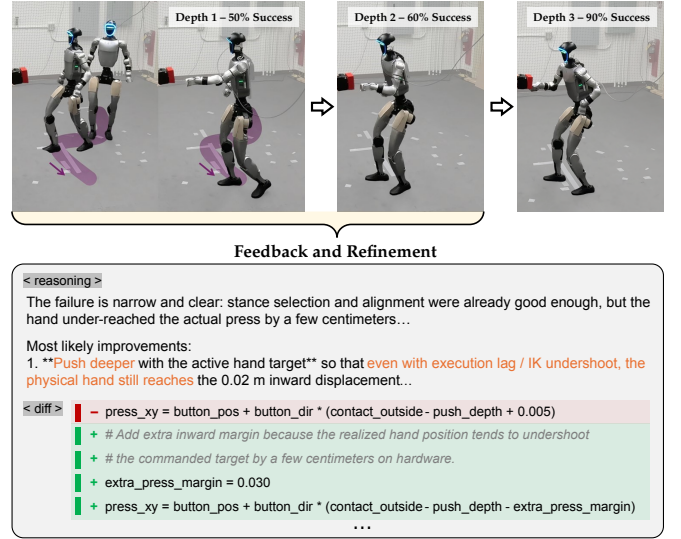


Fig. 8: Example of one refinement depth on hardware, generated by the LLM without any human feedback. Top: executions at refinement depths 1 to 3. Bottom: at depth 1, the rollout feedback attributes the failure to the end-effector under-reaching the button, and the resulting code change introduces an explicit inward margin to compensate for the undershoot.

tasks improve the code the LLM writes for a new one. The generated policies could also supervise a vision-based policy by distillation, or seed an RL replay buffer.

ACKNOWLEDGMENT

The authors thank Lasse Peters and Hongrui Zhao for their feedback on the manuscript, and Joohwan Seo for his feedback and for the end-effector design used on the Unitree G1.

REFERENCES

- [1] Q. Liao *et al.*, “Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion,” *arXiv preprint arXiv:2508.08241*, 2025.
- [2] I. Radosavovic *et al.*, “Learning humanoid locomotion over challenging terrain,” in *Conference on Robot Learning (CoRL)*, 2024.
- [3] S. Zhao *et al.*, “Resmimic: From general motion tracking to humanoid whole-body loco-manipulation via residual learning,” *arXiv preprint arXiv:2510.05070*, 2025.
- [4] Y. Kuang *et al.*, “Skillblender: Towards versatile humanoid whole-body loco-manipulation via skill blending,” *arXiv preprint arXiv:2506.09366*, 2025.
- [5] J. Li *et al.*, “Accelerating and scaling mpc-guided reinforcement learning for humanoid locomotion and manipulation,” *arXiv preprint arXiv:2606.05687*, 2026.
- [6] S. Bohez *et al.*, “Imitate and repurpose: Learning reusable robot movement skills from human and animal behaviors,” *arXiv preprint arXiv:2203.17138*, 2022.
- [7] Y. J. Ma *et al.*, “Eureka: Human-level reward design via coding large language models,” in *International conference on learning Representations*, vol. 2024, pp. 26516–26560, 2024.
- [8] M. Seo *et al.*, “Deep imitation learning for humanoid loco-manipulation through human teleoperation,” in *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, pp. 1–8, IEEE, 2023.
- [9] H. Weng *et al.*, “Hdmi: Learning interactive humanoid whole-body control from human videos,” *arXiv preprint arXiv:2509.16757*, 2025.
- [10] T. Xie *et al.*, “Grail: Generating humanoid loco-manipulation from 3d assets and video priors,” *arXiv preprint arXiv:2606.05160*, 2026.
- [11] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500, IEEE, 2023.

- [12] L. Fu *et al.*, “Cap-x: A framework for benchmarking and improving coding agents for robot manipulation,” *arXiv preprint arXiv:2603.22435*, 2026.
- [13] K. Chen *et al.*, “Gap: A graph-as-policy multi-agent self-learning harness for variational automation tasks,” *arXiv preprint arXiv:2607.05369*, 2026.
- [14] Y.-J. Wang *et al.*, “Prompt a robot to walk with large language models,” *arXiv preprint arXiv:2309.09969*, 2023.
- [15] Anthropic, “How Claude performs on robotics tasks.” <https://www.anthropic.com/research/claude-plays-robotics>, 2026.
- [16] L. Yang *et al.*, “Handoff: Humanoid agentic task-space whole-body control via distilled complementary teachers,” *arXiv preprint arXiv:2606.06493*, 2026.
- [17] Z. Luo *et al.*, “Sonic: Supersizing motion tracking for natural humanoid whole-body control,” *Science Robotics*, vol. 11, no. 117, p. eaed4592, 2026.
- [18] B. Romera-Paredes *et al.*, “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.
- [19] H. Ye *et al.*, “Reevo: Large language models as hyper-heuristics with reflective evolution,” *Advances in neural information processing systems*, vol. 37, pp. 43571–43608, 2024.
- [20] A. Novikov *et al.*, “Alphaevolve: A coding agent for scientific and algorithmic discovery,” *arXiv preprint arXiv:2506.13131*, 2025.
- [21] A. Karpathy, “Autoresearch.” <https://github.com/karpathy/autoresearch>, 2026. GitHub repository.
- [22] K. Ryu *et al.*, “Curriculum: Automatic task curricula design for learning complex robot skills using large language models,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4470–4477, IEEE, 2025.
- [23] S. Choi *et al.*, “Craft: Coaching reinforcement learning autonomously using foundation models for multi-robot coordination tasks,” *arXiv preprint arXiv:2509.14380*, 2025.
- [24] Y. Du *et al.*, “Vision-language models as success detectors,” *arXiv preprint arXiv:2303.07280*, 2023.
- [25] A. Sygkounas *et al.*, “Memento: Memory-guided memetic code-as-policy evolution,” *arXiv preprint arXiv:2607.22832*, 2026.
- [26] J. Wang *et al.*, “Autonomous behavior planning for humanoid loco-manipulation through grounded language model,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2024.
- [27] C. Wen *et al.*, “Humanoid agent via embodied chain-of-action reasoning with multimodal foundation models for zero-shot loco-manipulation,” *arXiv preprint arXiv:2504.09532*, 2025.
- [28] M. Schuck *et al.*, “Learning loco-manipulation from smpc demonstrations with sparse offline-to-online rl,” *arXiv preprint arXiv:2608.12063*, 2026.
- [29] I. Taouil *et al.*, “Motiondisco: Motion discovery for extreme humanoid loco-manipulation,” *arXiv preprint arXiv:2606.06139*, 2026.
- [30] M. Hosseinzadeh *et al.*, “Kite: Keyframe-indexed tokenized evidence for vlm-based robot failure analysis,” *arXiv preprint arXiv:2604.07034*, 2026.
- [31] H. Liu *et al.*, “Ego-vision world model for humanoid contact planning,” *arXiv preprint arXiv:2510.11682*, 2025.
- [32] J. Schulman *et al.*, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.